

INTRODUCCIÓN A ARM CORTEX-M

1

El grupo de núcleos de procesadores Arm®Cortex®-M es una serie de núcleos optimizados para la eficiencia de energía y operaciones determinísticas. Es ampliamente usado en microcontroladores (MCUs) y también puede ser encontrado embebidos dentro de microprocesadores multi núcleo (MPUs).

Desde los microcontroladores iniciales con núcleo Cortex®-M3 y la introducción de variantes de núcleos optimizados para **baja potencia**: como el Cortex®-M0 y el Cortex®-M0+, núcleos optimizados para un mayor **rendimiento**: como el Cortex®-M7, núcleos para aplicaciones en **tiempo real** como el Cortex®-M4 o enfocados en **seguridad** como el Cortex®-M33. La arquitectura Arm®Cortex®-M es la arquitectura estándar de facto para MCUs de 32 bits de propósito general.



Las principales ventajas de tener la arquitectura estandarizada Arm®Cortex®-M son:

- Los ingenieros pueden transferir fácilmente su código de una serie de MCU a otra. Pueden elegir la correcta compensación entre eficiencia energética, rendimiento computacional, seguridad y un amplio rango de periféricos para sus sistemas.
- Aprovechar un rico ecosistema de proveedores de microcontroladores y de proveedores de herramientas de software y hardware.
- Acelerar la innovación en aplicaciones embebidas.

STMicroelectronics fue uno de los primeros en adoptar los núcleos Arm®Cortex®-M y actualmente lidera el mercado con la mayor cartera de MCUs Cortex®-M de 32 bits.

1.1 ARM CORTEX®-M3

El núcleo de procesador de 32 bits Arm®Cortex®-M3 está diseñado para aplicaciones de costo limitado de **alto rendimiento** y **procesamiento en tiempo real** y capaz de manejar tareas complejas. Cualquier microcontrolador Arm ®Cortex®-M3 ofrece una alta escalabilidad combinada con combinado con un balance óptimo entre rendimiento y costo.

FUNDAMENTOS DEL STM32

2

2.1 TIMERS E INTERRUPCIONES

Los timers son una de las características más importantes en los microcontroladores modernos. Nos permiten medir cuánto tiempo toma en ejecutarse cierta tarea, controlar con precisión la sincronización de un pin e incluso correr sistemas operativos.

La línea de microcontroladores STM32 de STMicroelectronics no son la excepción: cada controlador ofrece un conjunto completo de timers para su uso.

2.1.1 Timers basics

Un timer (a veces referido como contador) es una pieza especial de hardware dentro de la mayoría de microcontroladores. Su función es simple: estos cuentan (en forma ascendente o descendente, dependiendo de su configuración). Por ejemplo, un timer de 8 bits contará desde 0 hasta 255. La mayoría de los timers se "reiniciarán" una vez que alcanzan su valor máximo, así, nuestro contador de 8 bits iniciaría de nuevo desde 0 una vez que alcance el valor de 255.

Es posible aplicar una variedad de configuraciones a la mayoría de los timers para cambiar la forma en que estos funcionan. Estas configuraciones usualmente se aplican en registros de funciones especiales dentro del microcontrolador. Por ejemplo, en lugar de contar hasta un máximo de 255, le puedes indicar al timer que quieres reiniciar la cuenta una vez que llegue a 100. Adicionalmente, puedes conectar más hardware o periféricos dentro del microcontrolador al timer como el alternar automáticamente el estado de un pin cuando el timer llegue a la cuenta indicada.

Alguna de las funciones que se pueden lograr con un timer son:

- Output Compare (OC): alternar el estado de un pin cuando el timer alcanza un cierto valor
- Input Capture (IC): medir el número de conteos de un timer entre eventos sobre un pin
- Pulse Width Modulation (PWM): alternar el estado de un pin cuando el timer alcanza un cierto valor y al reiniciar. Ajustando el tiempo de encendido vs el tiempo de apagado (duty cycle) se puede controlar efectivamente la cantidad de energía que pasará a través de otro dispositivo.

Prescaler

¿Qué tan rápido se ejecutan los timers? Depende de qué tan rápido le indique uno. Todos los timers requieren de algún tipo de reloj. La mayoría estarán conectados al reloj principal de la CPU del microcontrolador (otros, como los relojes en tiempo real (real time clock) tienen sus propias fuentes de reloj). Un timer marcará (incrementará en uno) cada vez que reciba un pulso del reloj.

Supongamos que tenemos un reloj principal (HCLK) de 80 MHz, y podríamos tener un timer ejecutándose a 80 MHz, pero esto puede ser demasiado rápido para nuestras aplicaciones. Un timer de 16 bits puede contar hasta $2^{16} = 65,535$ antes de que se reinicie, esto significa que podemos medir eventos no más grandes que $\frac{1}{80 \text{ MHz}} \times 65535 = 819 \mu\text{s}$.

Si deseamos medir tiempos más largos, necesitamos usar un prescaler, el cual es una pieza de hardware que divide los pulsos de la fuente de reloj. Por ejemplo, un prescaler de 80 convertiría un reloj de 80 MHz en un reloj de 1 MHz

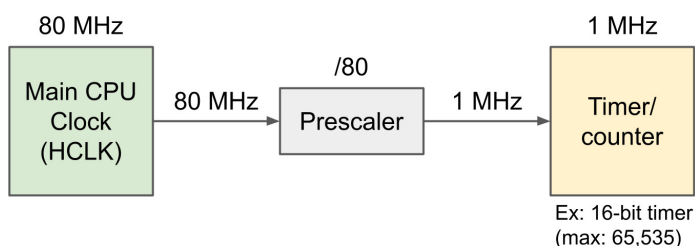


Figure 2.1: Uso del prescaler para disminuir los pulsos del reloj

Ahora nuestro timer daría un pulso cada $\frac{1}{1 \text{ MHz}} = 1 \mu\text{s}$ y asumiendo un timer de 16 bits entonces sería capaz de cronometrar eventos de hasta un máximo de $\frac{1}{1 \text{ MHz}} \times 65535 = 65.5 \text{ ms}$.

Escogiendo un timer

Si observas el datasheet de tu microcontrolador, encontrarás una sección hablando acerca de los timers disponibles que éste posee. Por ejemplo, en la hoja de datos del microcontrolador STM32F303RE, la sección 3.18 nos proporciona información de los timers a nuestra disposición.

Table 5. Timer feature comparison

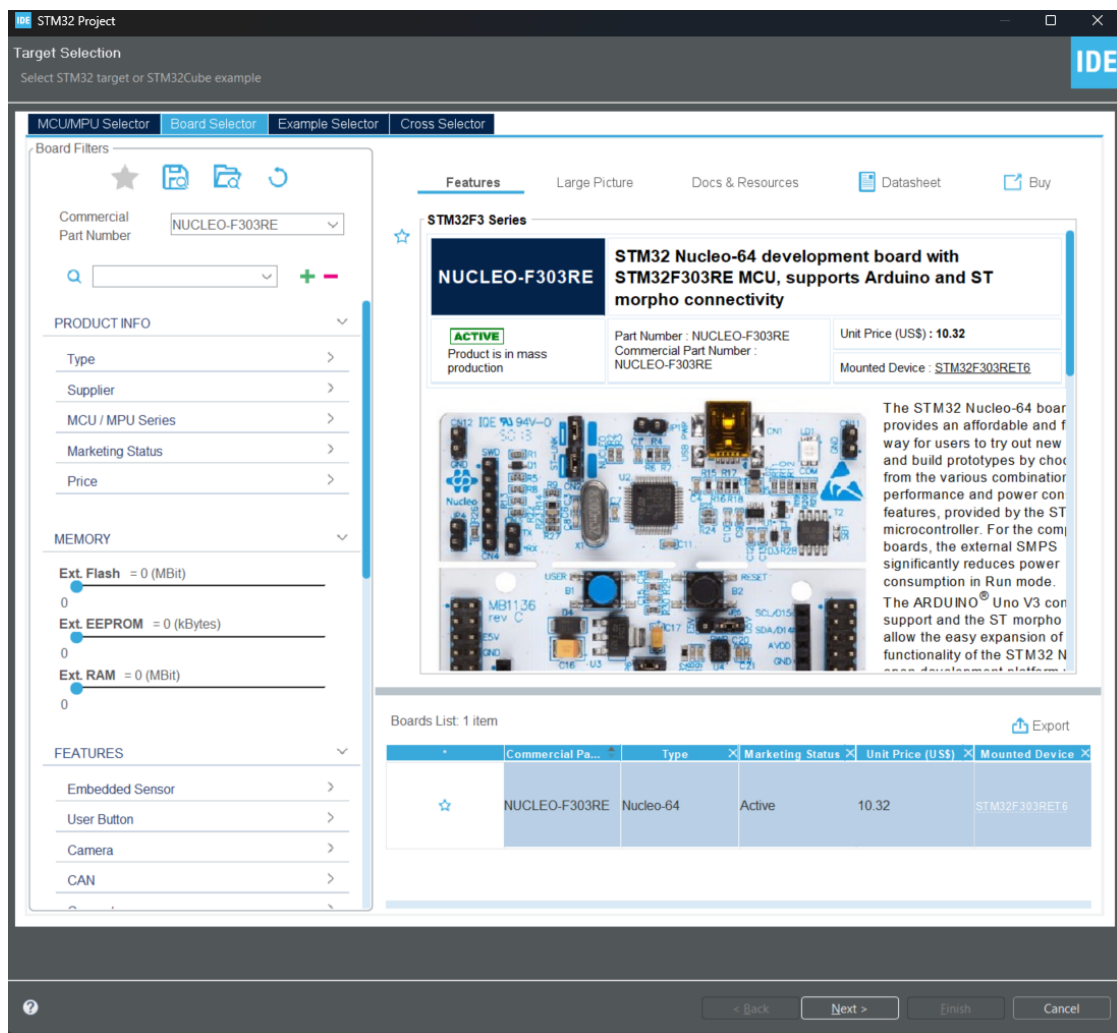
Timer type	Timer	Counter resolution	Counter type	Prescaler factor	DMA request generation	Capture/compare channels	Complementary outputs
Advanced	TIM1, TIM8, TIM20	16-bit	Up, Down, Up/Down	Any integer between 1 and 65536	Yes	4	Yes
General-purpose	TIM2	32-bit	Up, Down, Up/Down	Any integer between 1 and 65536	Yes	4	No
General-purpose	TIM3, TIM4	16-bit	Up, Down, Up/Down	Any integer between 1 and 65536	Yes	4	No
General-purpose	TIM15	16-bit	Up	Any integer between 1 and 65536	Yes	2	1
General-purpose	TIM16, TIM17	16-bit	Up	Any integer between 1 and 65536	Yes	1	1
Basic	TIM6, TIM7	16-bit	Up	Any integer between 1 and 65536	Yes	0	No

2.1.2 LEDs blinking usando timers

En este ejercicio de práctica se utilizan los timers TIM2, TIM3 y TIM4 para hacer parpadear tres LEDs a diferentes frecuencias. Dichas frecuencias serán establecidas en 0.5 s (2 Hz), 1 s (1 Hz) y 2 s (0.5 Hz) respectivamente.

Nuevo proyecto

Una vez abierto nuestro IDE, en la pestaña de **Board selector** buscamos la tarjeta de desarrollo NUCLE0-F303RE creamos un nuevo proyecto con el nombre de LEDs_Blinking_With_Timers



En la ventana de **Device Configuration Tool** podremos encontrar una figura del microcontrolador utilizado con todos los pines disponibles para ser configurados a nuestro criterio. Dentro de la sección de Timers seleccionamos el Timer2. los parámetros a configurar dentro de la subsección de **Mode** son:

- **Clock source:** Establecerla como *Internal Clock*
- **Channel1:** Establecerlo como *PWM Generation CH1*

Nuestra meta es generar una señal de reloj con una frecuencia de 0.5 Hz. Nuestro reloj principal tiene una frecuencia de 72 MHz, sin embargo, esta es una frecuencia demasiado grande para nuestros propósitos.

Para bajar dicha frecuencia de nuestro reloj principal a tan sólo 0.5 Hz haremos uso del **prescaler** y del **Autoreload register**, parámetros que se encuentran en la subsección de **Configuration**.

